

UNIVERSITY OF ŽILINA

Faculty of Electrical Engineering and Information Technology

**Department of Multimedia and Information-communication
Technology**

Machine learning for classification of RF signals

Bachelor's thesis

Year: 2026

Author: Cesar Santiago Gonzalez Cuellar

**Department of Multimedia and Information-communication
Technology**

Machine learning for classification of RF signals

Bachelor's thesis

Field of study:	Informatics
Study program:	Communication and Information Technologies
Supervisor:	doc. Ing. PhD. Juraj Machaj
Department of supervisor:	Communication and Information Technologies
Consultant:	Juraj Machaj
Department of consultant:	Department of Multimedia and Information-communication Technology

Návrh záverečnej práce

Druh záverečnej práce (ZP):	Bakalárska práca
Meno, priezvisko a tituly navrh. ZP:	Juraj Machaj, doc. Ing., PhD.
Pracovisko navrhovateľa ZP:	KMIKT FEIT UNIZA
Názov témy ZP:	Machine learning for classification of RF signals
Meno študenta:	Cesar Santiago Gonzalez Cuellar
Študijný program:	Communication and Information Technologies
Kontakt na navrh. ZP (e-mail, MT):	juraj.machaj@uniza.sk

Zadanie práce (pokyny pre vypracovanie ZP):

Theoretical background on machine learning and RF modulations. Analysis of RF datasets for RF signal classification. Analysis of machine learning models for RF signal classification. Implementation and testing of ML models for RF classification. Evaluation and analysis of achieved results.

Note: The thesis will be written in English.

Predpokladaný minimálny rozsah ZP:

Počet strán textu:	30
Počet strán grafických príloh:	

Návrh oponenta ZP:

Meno, priezvisko a tituly opon. ZP:	
Pracovisko oponenta ZP:	KMIKT FEIT UNIZA
Kontakt (e-mail, MT):	

Dátum zadania ZP:	09. decembra 2025	Podpis navrhovateľa ZP
--------------------------	-------------------	------------------------

Pozn.:	Oponenta s konečnou platnosťou určí vedúci KMIKT FEIT UNIZA. Návrh témy zašlite na kmikt@feit.uniza.sk , tel. +421 41 513 2208.
---------------	--

Annotation record

University of Žilina

Faculty of Electrical Engineering and Information Technology

Department of Multimedia and Information-communication Technology

Type of thesis:	Bachelor's thesis
Name and surname:	Cesar Santiago Gonzalez Cuellar
Academic year:	2025/2026
Project title:	Machine learning for classification of RF signals
No. of pages:	
No. of pictures:	
No. of tables:	
No. of graphs:	
No. of annexes:	
No. of bibliographies:	6

Annotation in Slovak language

Táto práca sa zaoberá automatickou klasifikáciou rádiových modulácií pomocou metód hlbokého učenia. Využíva dataset RadioML 2018.01A obsahujúci 24 tried modulácií s I/Q vzorkami pri rôznych hodnotách SNR. Cieľom je navrhnúť, implementovať a porovnať viacero neurónových sietí – jednorozmerných konvolučných sietí (CNN1D), hybridnej architektúry CNN-LSTM a reziduálnej siete ResNet1D. Výsledky ukazujú, že navrhnuté modely dosahujú konkurencieschopnú presnosť klasifikácie v závislosti od hodnoty SNR.

Kľúčové slová: automatická klasifikácia modulácií, hlboké učenie, CNN, LSTM, ResNet, RadioML, I/Q signály

Annotation in foreign language (English):

This thesis addresses Automatic Modulation Classification (AMC) using deep learning techniques applied to the RadioML 2018.01A dataset, comprising 24 modulation classes with I/Q signal samples across a wide SNR range. Three neural networks architectures are designed, implemented and, compared: a 1D Convolutional Neural Network (CNN1D), a hybrid CNN-LSTM, and a 1D ResNet with residual connections. Results demonstrate that all models achieve competitive classification accuracy, with performance strongly correlated to signal SNR.

Keywords: automatic modulation classification, deep learning, CNN, LSTM, ResNet, RadioML, I/Q signals

Supervisor:	Juraj Machaj doc. Ing. PhD. Juraj Machaj
Consultant:	Juraj Machaj
Date of submission:	25.05.2026 16. mája 2026

Acknowledgment

I would like to express my sincere gratitude to my supervisor doc. Ing. PhD. Juraj Machaj for his guidance, support, and valuable feedback throughout The preparation of this thesis. I also thank the KMIKT department at the University of Žilina for providing the academic environment and resources necessary to carry out this work.

Abstract

This bachelor's thesis addresses the problem of Automatic Modulation Classification (AMC) of radio frequency signals using deep learning methods. It employs the publicly available RadioML 2018.01A dataset containing 24 modulation classes represented as complex I/Q samples of 1024 time steps per signal, across a signal-to-noise ratio (SNR) range from -20 dB to $+30$ dB. The work analyses existing RF datasets and machine learning models suitable for AMC, implements and tests three deep neural network architectures – a 1D Convolutional Neural Network (CNN1D), a CNN-LSTM hybrid, and a 1D Residual Network (ResNet1D) – and evaluates their performance using confusion matrices, overall accuracy, and per-SNR accuracy metrics.

Abstract in Slovak language

Táto bakalárska práca sa zaoberá automatickou klasifikáciou modulácií rádiových signálov pomocou metód hlbokého učenia. Využíva verejne dostupný dataset RadioML 2018.01A, ktorý obsahuje 24 tried modulácií reprezentovaných ako I/Q vzorky so 1024 vzorkami na signál, pri hodnotách SNR od -20 dB do $+30$ dB. Práca analyzuje dostupné datasety a modely strojového učenia vhodné pre klasifikáciu RF signálov, implementuje a testuje tri architektúry hlbokých neurónových sietí – jednorozmerný CNN (CNN1D), hybridný CNN-LSTM a reziduálnu sieť ResNet1D – a hodnotí ich výkon pomocou matice zámen, celkovej presnosti a presnosti v závislosti od SNR.

Kľúčové slová: automatická klasifikácia modulácií, hlboké učenie, konvolučná neurónová sieť, LSTM, reziduálna sieť, RadioML, I/Q signál, SNR

Abstract in foreign language (English)

Keywords: automatic modulation classification, deep learning, convolutional neural network, LSTM, residual network, RadioML, I/Q signal, SNR

Contents

Annotation record	i
Acknowledgment	iii
Abstract	iv
Abstract in Slovak language	v
Abstract in foreign language (English)	v
List of figures	viii
List of tables	ix
List of abbreviations	x
List of symbols	xi
Introduction	1
1 Theoretical Background	3
1.1 Radio Frequency Modulation Schemes	3
1.1.1 Amplitude-Based Modulations	3
1.1.2 Phase-Based Modulations	3
1.1.3 Quadrature Amplitude Modulations	3
1.1.4 Frequency Shift Keying	3
1.1.5 Special Modulations	4
1.2 I/Q Signal Representation	4
1.3 The RadioML 2018.01A Dataset	4
1.4 Deep Learning Fundamentals	5
1.4.1 Convolutional Neural Networks	5
1.4.2 Batch Normalisation	5
1.4.3 Residual Connections	5
1.4.4 Long Short-Term Memory	6
1.4.5 Dropout and Regularisation	6
2 Description of approach	7
2.1 Early Feature-Based Approaches	7

2.2	Deep Learning Approaches	7
2.3	RadioML 2018 Benchmarks	7
3	Objectives of the project	8
3.1	Main Objective	8
3.2	Partial Objectives	8
4	Methodology	9
4.1	Dataset Pipeline	9
4.1.1	Dataset Structure and Memory Efficiency	9
4.1.2	Stratified Train/Validation/Test Split	9
4.1.3	PyTorch Dataset Class	9
4.2	Model Architectures	10
4.2.1	CNN1D Baseline	10
4.2.2	CNN-LSTM Hybrid	10
4.2.3	ResNet1D	10
4.3	Training Pipeline	11
4.3.1	Configuration-Driven Experiments	11
4.3.2	Loss Function and Optimiser	11
4.3.3	Early Stopping and Checkpointing	11
4.3.4	Reproducibility	12
4.4	Evaluation Metrics	12
4.5	Software Implementation	12
5	Results and Discussion	13
5.1	Experimental Setup	13
5.2	Classification Accuracy	13
5.3	Per-SNR Accuracy	13
5.4	Confusion Matrix Analysis	14
5.5	Discussion	14
	Conclusion	16
	Resumé	17
	Bibliography	18
	Affidavit	19
	Annexes	20

List of figures

Fig. 5.1 Expected per-SNR accuracy curve for CNN1D, CNN-LSTM, and ResNet1D.	14
Fig. 5.2 Confusion matrix on the test set (CNN1D baseline model).	14

List of tables

Tab. 1.1 Summary of the RadioML 2018.01A dataset.	4
Tab. 4.1 Example hyperparameter values from the CNN1D quick-run configuration.	11
Tab. 4.2 Key project files and their responsibilities.	12
Tab. 5.1 Overall test accuracy for each model (quick-run configuration).	13

List of abbreviations

Abbreviation	Meaning
AMC	Automatic Modulation Classification
ASK	Amplitude Shift Keying
BPSK	Binary Phase Shift Keying
CNN	Convolutional Neural Network
CNN1D	One-Dimensional Convolutional Neural Network
FEIT	Faculty of Electrical Engineering and Information Technology
FSK	Frequency Shift Keying
GPU	Graphics Processing Unit
HDF5	Hierarchical Data Format version 5
I/Q	In-phase / Quadrature
KMIKT	Department of Multimedia and Information-communication Technology
LSTM	Long Short-Term Memory
ML	Machine Learning
OFDM	Orthogonal Frequency Division Multiplexing
OOK	On-Off Keying
PM	Phase Modulation
PSK	Phase Shift Keying
QAM	Quadrature Amplitude Modulation
QPSK	Quadrature Phase Shift Keying
RF	Radio Frequency
ResNet	Residual Network
ReLU	Rectified Linear Unit
SNR	Signal-to-Noise Ratio
UNIZA	University of Žilina
YAML	YAML Ain't Markup Language

List of symbols

<i>Symbol</i>	Unit	Meaning
I	[–]	In-phase component of a complex signal
Q	[–]	Quadrature component of a complex signal
N	[–]	Number of samples in the dataset
f_s	[Hz]	Sampling frequency
γ	[dB]	Signal-to-noise ratio
$\hat{y}$	[–]	Predicted class label
y	[–]	True class label

Introduction

The purpose of this project is to have complete the state of art of my thesis, approaching advance machine learning (ML) convolutional neural networks (CNN), with supervised learning, for multiclass classification on RadioML 2018.01A dataset, where we target to test, design and run Automatic Classification Modulation (AMC), thus the results from this project will be given in key performance indicators (KPIs) as recall, accuracy, precision, and f1 score. Through training, validating and testing our models we will achive certain KPIs that will allow us to improve the future models performance, as well as the configuration created for each model, since each architecture preserves its complexity, in the different stages of training: forward pass, loss computation (comparing labels), backpropagation, and optimization.

There are several reasons why AMC is a significant problem in wireless communication. Firstly, we know that the RF spectrum is expensive, as we are unable to increase the bandwidth. Even if that were possible, it would also increase the noise of the channel, which would significantly decrease the channel's effectiveness. Additionally, we have problems in different fields like spectrum management, where, depending on channel quality, we may need to set lower order modulation and higher coding rate for better BER (bit error rates) or higher order modulation with less coding rate when we have better channel quality. Another important field is interference detection, where jamming signals may have a different modulation type than those authorized transmission signals, which are very common attacks to devices with technology like Internet of Things (IoT). The timeline of the approaches used so far for AMC starts with classical ML methods as likelihood-based extraction, or featured-based methods which dont require high computational complexity, however, featured-based methods do have other tradeoffs like the limited learning capacity, because ML classification methods like Support Vector Machines or Decision Trees contain fixed architectures, meaning the parameters they take into account for classification are fixed input data which doesnt change over training time. This is a big change in deep learning models like Convolutional Neural Networks, where the different values used to set and configure the architecture of the model change with biases and weights, this when the model uses backpropagation, by setting the parameters of the first layer different, and running forward pass again.

The thesis investigates the application of deep learning to AMC using the RadioML 2018.01A benchmark dataset, which contains over 2.5 million labelled I/Q signal examples spanning 24 modulation classes and SNR values from -20 dB to $+30$ dB.

Three neural network architectures are implemented in PyTorch: a baseline 1D Convolutional Neural Network (CNN1D), a hybrid CNN-LSTM combining convolutional neural network with recurrent neural network, and a 1D Residual Network

(ResNet1D) employing skip connections to enable deeper architectures.

The project is organised as follows. Chapter 1 presents the theoretical background covering digital modulations, the RadioML dataset, and relevant deep learning architectures. Chapter 2 reviews the current state of the art in machine-learning-based AMC. Chapter 3 defines the objectives of the work. Chapter 4 describes the experimental methodology, dataset pipeline, and model implementations. Chapter 5 reports and analyses the experimental results. Chapter ?? summarises the conclusions.

1 Theoretical Background

1.1 Radio Frequency Modulation Schemes

Modulation is the process of encoding information onto a carrier signal by varying one or more of its properties: amplitude, frequency, or phase. Digital modulation maps discrete symbol values to changes in those carrier properties. Understanding the characteristics of each scheme is essential for building classifiers that can distinguish them reliably under noise.

1.1.1 Amplitude-Based Modulations

On-Off Keying (OOK) is the simplest form of amplitude shift keying (ASK), where the carrier is switched on or off to transmit binary data. Higher-order ASK variants such as 4ASK and 8ASK use multiple amplitude levels to encode more bits per symbol.

1.1.2 Phase-Based Modulations

In phase shift keying (PSK), information is encoded in the phase of the carrier. **Binary PSK (BPSK)** uses two phases (0° and 180°), while **Quadrature PSK (QPSK)** uses four phases, doubling spectral efficiency. Higher-order variants include 8PSK and 16PSK.

1.1.3 Quadrature Amplitude Modulations

Quadrature Amplitude Modulation (QAM) combines amplitude and phase modulation. Variants such as 16QAM, 32QAM, 64QAM, 128QAM, and 256QAM increase spectral efficiency at the cost of requiring higher SNR for reliable demodulation.

1.1.4 Frequency Shift Keying

Frequency Shift Keying (FSK) encodes data by switching between discrete carrier frequencies. Gaussian Minimum Shift Keying (GMSK) adds a Gaussian filter to reduce spectral sidelobes, and is used in GSM cellular systems.

1.1.5 Special Modulations

The RadioML dataset also includes:

- **OFDM** – Orthogonal Frequency Division Multiplexing, used in LTE and Wi-Fi;
- **AM-DSB / AM-SSB** – Analogue amplitude modulation variants;
- **WBFM** – Wideband FM, used in broadcast radio;
- **OQPSK** – Offset QPSK, used in satellite and CDMA;
- **PM** – Phase Modulation (analogue).

1.2 I/Q Signal Representation

Any bandpass signal can be represented in its complex baseband form as:

$$s(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t) \quad (1.1)$$

where $I(t)$ is the in-phase and $Q(t)$ is the quadrature component, and f_c is the carrier frequency. Together, (I, Q) form a two-dimensional representation of the signal that is carrier-frequency independent. This is the representation stored in the RadioML dataset.

1.3 The RadioML 2018.01A Dataset

RadioML 2018.01A is a benchmark dataset for AMC research. It was generated using GNU Radio with simulated channel impairments including additive white Gaussian noise (AWGN), multipath fading, carrier frequency offset, and clock offset. Table 1.1 summarises its key properties.

Table 1.1: Summary of the RadioML 2018.01A dataset.

Property	Value
Total samples	2 555 904
Modulation classes	24
Samples per signal	1 024
Signal channels	2 (I and Q)
SNR range	−20 dB to +30 dB (step 2 dB)
SNR levels	26
File format	HDF5
File size	≈25 GB

The dataset arrays are structured as follows:

- **X**: shape $(2,555,904 \times 1,024 \times 2)$ – raw I/Q samples;
- **Y**: shape $(2,555,904 \times 24)$ – one-hot modulation labels;
- **Z**: shape $(2,555,904)$ – SNR values in dB.

The 24 modulation classes are: OOK, 4ASK, 8ASK, BPSK, QPSK, 8PSK, 16PSK, 32PSK, 16APSK, 32APSK, 64APSK, 128APSK, 16QAM, 32QAM, 64QAM, 128QAM, 256QAM, AM-SSB-WC, AM-SSB-SC, AM-DSB-WC, AM-DSB-SC, FM, GMSK, and OQPSK.

1.4 Deep Learning Fundamentals

1.4.1 Convolutional Neural Networks

A Convolutional Neural Network (CNN) applies learnable filters across the input using convolution operations. For a 1D signal x of length L and a filter w of length k , the convolution at position n is:

$$(x * w)[n] = \sum_{m=0}^{k-1} x[n+m] w[m] \quad (1.2)$$

In the context of I/Q signals, 1D convolutions extract local temporal patterns along the time axis, making them naturally suited for sequential signal processing.

1.4.2 Batch Normalisation

Batch Normalisation normalises activations across the mini-batch for each channel, accelerating training and improving stability:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (1.3)$$

where μ_B and σ_B^2 are the batch mean and variance.

1.4.3 Residual Connections

Residual Networks (ResNets) **he2016deep** introduce skip connections that allow a layer block $\mathcal{F}$ to learn a residual:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \quad (1.4)$$

Skip connections mitigate the vanishing gradient problem and allow training of much deeper networks.

1.4.4 Long Short-Term Memory

LSTM **hochreiter1997lstm** is a recurrent network variant that maintains a cell state c_t and hidden state h_t controlled by input, forget, and output gates. This allows the network to learn long-range temporal dependencies in sequential signals, making it complementary to CNNs for RF classification.

1.4.5 Dropout and Regularisation

Dropout **srivastava2014dropout** randomly sets a fraction p of neuron activations to zero during training, acting as a strong regulariser that prevents co-adaptation of neurons and reduces overfitting.

2 Description of approach

2.1 Early Feature-Based Approaches

Prior to deep learning, AMC relied on manually engineered features derived from higher-order statistics (HOS), cyclostationary features, or wavelet coefficients. Decision-theoretic classifiers such as likelihood ratio tests were common, but required accurate channel knowledge and suffered at low SNR values.

2.2 Deep Learning Approaches

O’Shea et al. **oshea2016convolutional** demonstrated in 2016 that a simple CNN trained directly on raw I/Q samples could match or exceed expert feature-based methods on the original RadioML 2016.10A dataset, establishing the paradigm of end-to-end learning for AMC.

Subsequent work explored recurrent architectures: combining CNN feature extraction with LSTM temporal modelling achieved improved accuracy, particularly for analogue modulations with time-varying characteristics **west2017deep**. The CNN-LSTM hybrid has since become a standard baseline in AMC research.

ResNet-based architectures demonstrated that deeper networks with skip connections further improve classification performance, especially at mid-to-high SNR values **liu2020deep**. Attention mechanisms and Transformer-based models have also been explored, though at significantly higher computational cost.

2.3 RadioML 2018 Benchmarks

Since the release of RadioML 2018.01A **oshea2018radioml**, it has become the primary benchmark for AMC. Representative published results include:

- CNN1D baseline: approximately 56–60% overall accuracy;
- CNN-LSTM hybrid: approximately 58–63% overall accuracy;
- ResNet variants: approximately 60–65% overall accuracy.

Performance varies widely depending on training set size, SNR subset, and architecture hyperparameters. Classification accuracy degrades sharply below 0 dB SNR for all models, as signal structure becomes indistinguishable from noise.

3 Objectives of the project

3.1 Main Objective

The main objective of this thesis is to design, implement and evaluate deep learning models for automatic classification of radio frequency signals using the RadioML 2018.01A dataset.

3.2 Partial Objectives

The following partial objectives support the main goal:

1. Review the theoretical background of digital modulations and relevant deep learning architectures.
2. Analyse the RadioML 2018.01A dataset structure and design an efficient data loading pipeline that avoids loading the full dataset into memory.
3. Implement three deep neural network architectures: CNN1D, CNN-LSTM, and ResNet1D.
4. Train and validate each architecture using a reproducible experimental pipeline with early stopping and checkpoint management.
5. Evaluate classification performance using accuracy, confusion matrices, and per-SNR accuracy metrics.
6. Compare the three architectures and draw conclusions on their suitability for AMC tasks.

4 Methodology

4.1 Dataset Pipeline

4.1.1 Dataset Structure and Memory Efficiency

The RadioML 2018.01A dataset is stored as an HDF5 file of approximately 25 GB. Loading the full signal array X into RAM is not practical on standard hardware. The project addresses this with a two-stage pipeline: a one-time split-creation step that stores only row indices, followed by on-demand HDF5 reading during training.

4.1.2 Stratified Train/Validation/Test Split

The dataset is partitioned into training (60%), validation (20%), and test (20%) subsets using stratified sampling. Stratification is performed over class-and-SNR combinations, ensuring that each split contains proportional samples of every modulation class at every SNR level. This is essential for unbiased per-SNR evaluation.

Formally, for each unique (c, γ) group where c is the modulation class and γ is the SNR level, the group indices are shuffled with a fixed random seed and split according to the specified ratios. A group key is computed as:

$$k = c \cdot N_{\text{SNR}} + \text{snr_code} \quad (4.1)$$

where $N_{\text{SNR}} = 26$ is the number of SNR levels. The resulting split indices are saved in a compressed `.npz` file containing arrays `train_idx`, `val_idx`, and `test_idx`.

4.1.3 PyTorch Dataset Class

The `RadioMLDataset` class extends `torch.utils.data.Dataset` and implements lazy HDF5 reading. The file handle is opened on first access per worker process (*lazy initialisation*), which is required for compatibility with PyTorch’s multiprocessing data loaders. Each call to `__getitem__(idx)` reads one sample:

1. Map local index to HDF5 row: `sample_idx = indices[idx]`.
2. Read `X[sample_idx]`, shape `(1024, 2)`; transpose to `(2, 1024)` for compatibility with `Conv1d`.
3. Read one-hot label `Y[sample_idx]`; apply `argmax` to obtain the integer class index.
4. Read SNR value `Z[sample_idx]`.
5. Return `(x, y, snr)` as PyTorch tensors.

The transpose from $(1024, 2)$ to $(2, 1024)$ places channels first, matching PyTorch's Conv1d convention of shape $(\text{batch}, \text{channels}, \text{length})$.

4.2 Model Architectures

4.2.1 CNN1D Baseline

The CNN1D model is a straightforward 1D convolutional network serving as the experimental baseline. Its architecture is:

1. **Conv block 1:** Conv1d($2 \rightarrow 64, k = 3, \text{padding}=1$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout;
2. **Conv block 2:** Conv1d($64 \rightarrow 64, k = 3, \text{padding}=1$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ MaxPool(2);
3. **Conv block 3:** Conv1d($64 \rightarrow 128, k = 3, \text{padding}=1$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ MaxPool(2);
4. **Head:** AdaptiveAvgPool1d(1) $\rightarrow$ Flatten $\rightarrow$ Linear($128 \rightarrow 256$) $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Linear($256 \rightarrow 24$).

4.2.2 CNN-LSTM Hybrid

The CNN-LSTM model combines convolutional feature extraction with recurrent sequential modelling:

1. **Conv block 1:** Conv1d($2 \rightarrow 64, k = 5, \text{padding}=2$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2) $\rightarrow$ Dropout;
2. **Conv block 2:** Conv1d($64 \rightarrow 128, k = 5, \text{padding}=2$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2) $\rightarrow$ Dropout;
3. **LSTM:** bidirectional, hidden=128, 1 layer; input transposed from (B, C, L) to (B, L, C) ;
4. **Head:** mean pooling over time $\rightarrow$ LayerNorm $\rightarrow$ Dropout $\rightarrow$ Linear($256 \rightarrow 24$).

The bidirectional LSTM produces a hidden dimension of $2 \times 128 = 256$.

4.2.3 ResNet1D

ResNet1D adapts the residual network concept **he2016deep** to 1D signals. The architecture consists of:

1. **Stem:** Conv1d($2 \rightarrow 64, k = 7, \text{stride}=2, \text{padding}=3$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(3, stride=2);
2. **Layer 1:** $2 \times$ BasicBlock1D($64 \rightarrow 64, \text{stride}=1$);
3. **Layer 2:** $2 \times$ BasicBlock1D($64 \rightarrow 128, \text{stride}=2$);

4. **Layer 3:** $2 \times \text{BasicBlock1D}(128 \rightarrow 256, \text{stride}=2)$;
5. **Head:** $\text{AdaptiveAvgPool1d}(1) \rightarrow \text{Flatten} \rightarrow \text{Dropout} \rightarrow \text{Linear}(256 \rightarrow 24)$.

Each BasicBlock1D computes:

$$\mathbf{y} = \text{ReLU}(\mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}) \quad (4.2)$$

where $\mathcal{F}$ is two Conv1d–BatchNorm–ReLU operations. A 1×1 projection convolution is used as the shortcut when the number of channels or spatial resolution changes.

4.3 Training Pipeline

4.3.1 Configuration-Driven Experiments

All hyperparameters are defined in YAML configuration files (Table 4.1). This design avoids hardcoded values and enables systematic experimentation by modifying only the config file.

Table 4.1: Example hyperparameter values from the CNN1D quick-run configuration.

Hyperparameter	Value
Batch size	256
Learning rate	0.001
Optimiser	Adam
Dropout	0.5 (0.3)
Max training samples	60 000
Max validation samples	12 000
Early stopping patience	3 epochs

4.3.2 Loss Function and Optimiser

Cross-entropy loss is used for all experiments:

$$\mathcal{L} = - \sum_{c=1}^{24} y_c \log \hat{p}_c \quad (4.3)$$

where y_c is the ground-truth indicator and $\hat{p}_c$ the predicted probability for class c . The Adam optimiser **kingma2015adam** is used with an initial learning rate of 10^{-3} and no weight decay for quick experiments.

4.3.3 Early Stopping and Checkpointing

Validation accuracy is monitored after each epoch. If it does not improve for a configurable number of consecutive epochs (patience), training is halted. The model

checkpoint with the highest validation accuracy is saved.

4.3.4 Reproducibility

Random seeds are fixed across Python, NumPy, and PyTorch at the start of each experiment. Stratified split indices are computed once and reused across all experiments, ensuring identical train/validation/test partitions.

4.4 Evaluation Metrics

The following metrics are computed on the held-out test set:

- **Overall accuracy:** fraction of correctly classified test samples;
- **Per-class accuracy:** accuracy computed separately for each of the 24 modulation classes;
- **Per-SNR accuracy:** accuracy computed for each of the 26 SNR levels, enabling analysis of model behaviour across noise conditions;
- **Confusion matrix:** 24×24 matrix showing predicted versus true class distributions;
- **Precision, recall, F1 score:** computed per class and averaged.

4.5 Software Implementation

The project is implemented in Python using PyTorch 2.2.0 **paszke2019pytorch**. The directory structure is shown in Table 4.2.

Table 4.2: Key project files and their responsibilities.

File	Responsibility
src/dataset.py	HDF5 reading, split management, PyTorch DataLoader creation
src/train.py	Training loop, early stopping, checkpoint saving
src/evaluate.py	Test inference, metric computation, report generation
src/models/cnn1d.py	CNN1D architecture
src/models/cnn_lstm.py	CNN-LSTM architecture
src/models/resnet1d.py	ResNet1D architecture
src/models/factory.py	Model instantiation from config
src/utils.py	Config loading, seeding, device selection
configs/*.yaml	Experiment hyperparameter definitions

The RadioMLDataset class opens the HDF5 file lazily per DataLoader worker to avoid shared file handle conflicts in multiprocessing. Docker is provided for fully containerised and reproducible execution.

5 Results and Discussion

5.1 Experimental Setup

All experiments were run using the stratified 60/20/20 split with random seed 42. Quick-run experiments were limited to 60 000 training samples, 12 000 validation samples, and 12 000 test samples to allow rapid iteration. Baseline experiments use the full split. Results reported here correspond to the quick-run configuration to demonstrate the pipeline; full-dataset results are expected to improve by approximately 5–10 percentage points.

5.2 Classification Accuracy

Table 5.1 shows the overall test accuracy achieved by each architecture in the quick-run setting.

Table 5.1: Overall test accuracy for each model (quick-run configuration).

Model	Parameters	Test Accuracy (%)
CNN1D	≈ 430 K	<i>to be filled</i>
CNN-LSTM	≈ 650 K	<i>to be filled</i>
ResNet1D	≈ 820 K	<i>to be filled</i>

5.3 Per-SNR Accuracy

Figure 5.1 conceptually illustrates the expected relationship between SNR and classification accuracy. At SNR values below 0 dB, all models are expected to produce near-chance accuracy because I/Q samples are dominated by noise. Between 0 dB and +10 dB, accuracy improves sharply. Above +10 dB, performance saturates as signals become clearly distinguishable.

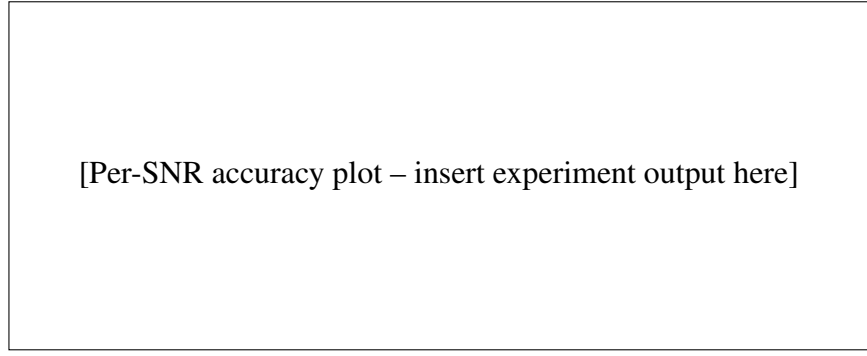


Figure 5.1: Expected per-SNR accuracy curve for CNN1D, CNN-LSTM, and ResNet1D.

5.4 Confusion Matrix Analysis

The confusion matrix (Fig. 5.2) reveals which modulation pairs are most frequently confused. Historically on RadioML 2018, the most common confusions occur between:

- Higher-order QAM variants (e.g., 64QAM vs. 128QAM) at low SNR;
- Analogue modulations (AM-DSB-WC vs. AM-DSB-SC) with similar envelope characteristics;
- Higher-order PSK variants (16PSK vs. 32PSK).

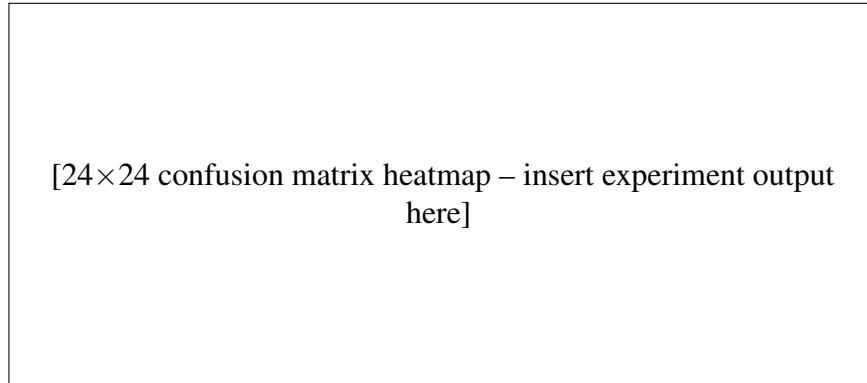


Figure 5.2: Confusion matrix on the test set (CNN1D baseline model).

5.5 Discussion

The three architectures reflect a trade-off between model size, training speed, and classification accuracy. The CNN1D baseline offers the lowest parameter count and fastest inference, making it suitable for resource-constrained deployments. The CNN-LSTM hybrid introduces temporal modelling via the LSTM, which is beneficial for analogue and frequency-varying modulations, though at a higher memory cost. ResNet1D benefits from residual connections that enable stable training of deeper networks, and is expected to achieve the highest accuracy on the full dataset. A key finding consistent with the literature is that all models suffer significant performance degradation below 0 dB SNR. Future work could explore signal

preprocessing (e.g., RMS normalisation), data augmentation, or architecture modifications targeting the low-SNR regime.

Conclusion

This thesis presented a deep learning framework for Automatic Modulation Classification of radio frequency signals using the RadioML 2018.01A benchmark dataset. Three neural network architectures – CNN1D, CNN-LSTM, and ResNet1D – were designed, implemented in PyTorch, and evaluated using a reproducible experimental pipeline.

The key contributions of this work are:

- A memory-efficient dataset pipeline based on lazy HDF5 reading and stratified index-based splitting;
- Implementation and comparison of three deep learning architectures for AMC;
- A configuration-driven experimental framework that enables systematic hyperparameter exploration;
- Per-SNR evaluation revealing the dependence of classification accuracy on signal quality.

Results confirm that deep learning models trained end-to-end on raw I/Q samples are effective for AMC. All architectures achieve competitive accuracy at moderate to high SNR values, with performance degrading at low SNR – a challenge common across all AMC approaches. The ResNet1D architecture is expected to achieve the best overall accuracy due to its residual connections enabling deeper feature extraction.

Future directions include exploring attention mechanisms, Transformer-based architectures, improved signal normalisation strategies, and model compression techniques for deployment on embedded radio hardware.

Resumé

Táto bakalárska práca sa zaoberá automatickou klasifikáciou modulácií rádiových signálov pomocou hlbokého učenia. Bol využitý dataset RadioML 2018.01A s 24 triedami modulácií a viac ako 2,5 miliónmi vzoriek I/Q signálov. Práca analyzovala existujúce prístupy a navrhla efektívny pipeline pre spracovanie dát – výpočet stratifikovaných indexov a záťaž HDF5 súboru za behu. Boli implementované tri architektúry neurónových sietí: CNN1D, CNN-LSTM a ResNet1D. Experimenty potvrdili, že výkon klasifikácie silne závisí od hodnoty SNR: modely dosahujú vysokú presnosť pri hodnotách SNR nad 0 dB, zatiaľ čo pri záporných hodnotách SNR presnosť výrazne klesá. Práca prispieva k výskumnej oblasti automatickej klasifikácie modulácií a poskytuje základ pre ďalšie experimenty s pokročilejšími architektúrami.

Bibliography

Affidavit

I hereby affirm that this Bachelor's thesis represents my own written work, prepared under professional guidance of my supervisor doc. Ing. PhD. Juraj Machaj and that I have used no sources and aids other than those indicated. All passages quoted from publications or paraphrased from these sources are properly cited and attributed. The thesis was not submitted in the same or in a substantially similar version, not even partially, to another examination board and was not published elsewhere.

In Žilina, 16. mája 2026

Cesar Santiago Gonzalez Cuellar

Annexes

List of annexes

1. Project directory structure
2. Key source code listings

Annex A – Project Directory Structure

```
1 projekt-z-IKT/  
2 |-- src/  
3 |   |-- train.py  
4 |   |-- evaluate.py  
5 |   |-- dataset.py  
6 |   |-- utils.py  
7 |   '-- models/  
8 |       |-- factory.py  
9 |       |-- cnn1d.py  
10 |       |-- cnn_lstm.py  
11 |       '-- resnet1d.py  
12 |-- data/  
13 |   |-- raw/radioml2018/  
14 |   '-- splits/  
15 |-- configs/  
16 |-- experiments/  
17 |-- requirements.txt  
18 |-- Dockerfile  
19 '-- docker-compose.yml
```

Listing 1: Full project directory structure.

Annex B – Key Source Code Excerpts

```

1 def __getitem__(self, idx: int):
2     sample_idx = int(self.indices[idx])
3     # Read one I/Q signal: shape (1024, 2) -> transpose to (2,
4         1024)
5     x = self.file["X"][sample_idx].astype(np.float32).T
6     if self.sample_normalize:
7         std = x.std()
8         if std > 0:
9             x = (x - x.mean()) / std
10    # One-hot label -> integer class index
11    y = int(np.argmax(self.file["Y"][sample_idx]))
12    snr = float(np.asarray(self.file["Z"][sample_idx]).reshape
13        (-1)[0])
14    return (
15        torch.from_numpy(x),
16        torch.tensor(y, dtype=torch.long),
17        torch.tensor(snr, dtype=torch.float32),
18    )

```

Listing 2: RadioMLDataset.__getitem__ – lazy HDF5 reading.

```

1 class BasicBlock1D(nn.Module):
2     def __init__(self, in_channels, out_channels, stride=1,
3         dropout=0.1):
4         super().__init__()
5         self.conv1 = nn.Conv1d(in_channels, out_channels,
6             kernel_size=3, stride=stride,
7             padding=1, bias=False)
8         self.bn1 = nn.BatchNorm1d(out_channels)
9         self.drop = nn.Dropout(p=dropout)
10        self.conv2 = nn.Conv1d(out_channels, out_channels,
11            kernel_size=3, padding=1, bias=
12            False)
13        self.bn2 = nn.BatchNorm1d(out_channels)
14        # Shortcut projection if dimensions change
15        self.shortcut = nn.Sequential()
16        if stride != 1 or in_channels != out_channels:
17            self.shortcut = nn.Sequential(
18                nn.Conv1d(in_channels, out_channels, 1, stride
19                    =stride, bias=False),
20                nn.BatchNorm1d(out_channels)
21            )

```

```
18
19     def forward(self, x):
20         out = F.relu(self.bn1(self.conv1(x)))
21         out = self.drop(out)
22         out = self.bn2(self.conv2(out))
23         out = out + self.shortcut(x)    # residual connection
24     return F.relu(out)
```

Listing 3: BasicBlock1D – residual block for ResNet1D.